EXPERIMENT 8

Demonstration of predictive models such as Decision Tree, Neural network and KNearest Neighbor using various domain datasets.

AIM

To implement and evaluate a Neural Network (MLPClassifier) on the Dry Bean Dataset and the Phishing Websites Dataset, and assess model performance using accuracy, classification report, confusion matrix, and ROC curves.

PROCEDURE

1. Install ucimlrepo library using pip.
2. Import ucimlrepo, pandas, matplotlib, sklearn modules.
3. Fetch the Dry Bean dataset (id=602) using fetch_ucirepo().
4. Extract features (X) and targets (y).
5. Encode target labels using LabelEncoder.
6. Split dataset into 80% train and 20% test sets.
7. Normalize features using StandardScaler.
8. Define MLPClassifier with layers (150, 100, 50), relu activation, adam solver.
9. Train the model using model.fit().
10. Predict on test set and compute accuracy.
11. Print classification report with target class names.
12. Plot confusion matrix using ConfusionMatrixDisplay.
13. Plot training loss curve.
14. Compute and plot ROC curves with AUC for each class.
15. Repeat steps 3–14 for the Phishing Websites dataset (id=327).

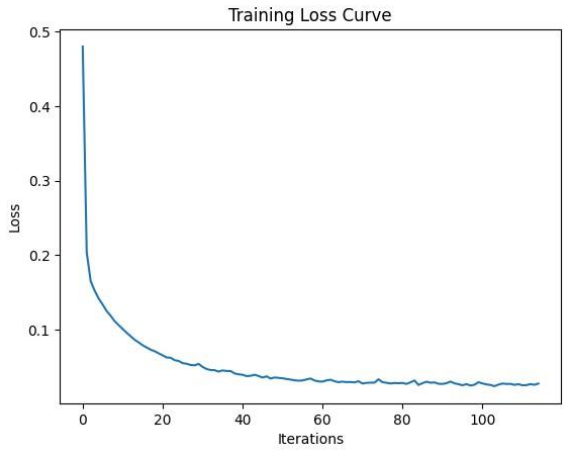
CODE

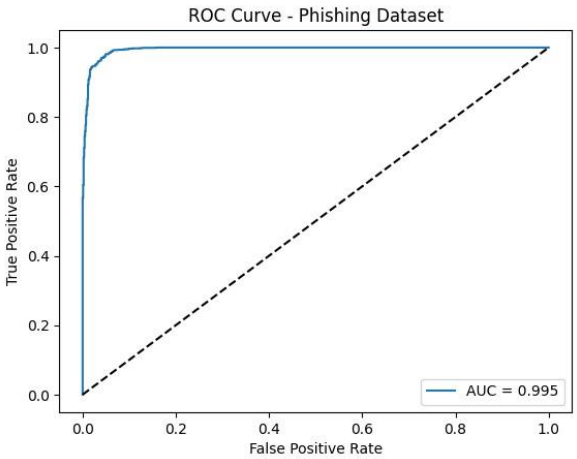
```
pip install ucimlrepo

from ucimlrepo import fetch_ucirepo import pandas as pd import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split from sklearn.preprocessing import
StandardScaler, LabelEncoder from sklearn.neural_network import MLPClassifier from
sklearn.metrics import classification_report, accuracy_score, ConfusionMatrixDisplay

# Dataset 1: Dry Bean
dry_bean = fetch_ucirepo(id=602) X
= dry_bean.data.features
y = dry_bean.data.targets

y = y.ravel() le =
LabelEncoder()
y = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = MLPClassifier(
    hidden_layer_sizes=(150, 100, 50),
activation='relu',
```

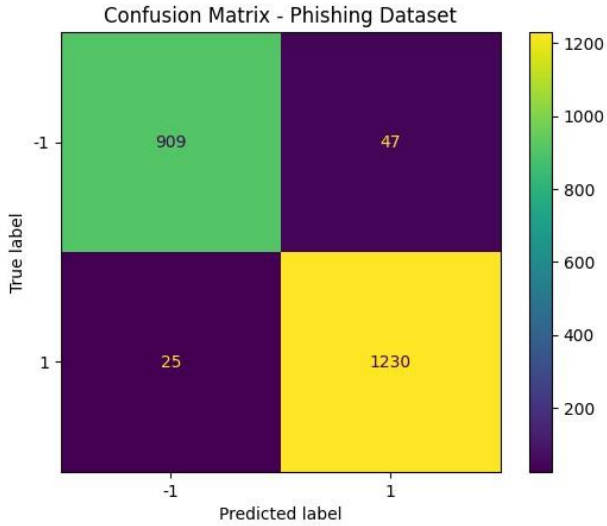
```
    solver='adam',
max_iter=500,
learning_rate_init=0.001,
random_state=42
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test) print('Accuracy:',
accuracy_score(y_test, y_pred))

target_names_list =
['Barbunya','Bombay','Cali','Dermosan','Horoz','Seker','Sira']
print(classification_report(y_test, y_pred, target_names=target_names_list))

ConfusionMatrixDisplay.from_estimator(model, X_test, y_test)
plt.title('Confusion Matrix - Dry Bean Dataset') plt.show()

plt.plot(model.loss_curve_)
plt.title('Training Loss Curve')
plt.xlabel('Iterations') plt.ylabel('Loss')
plt.show()
```

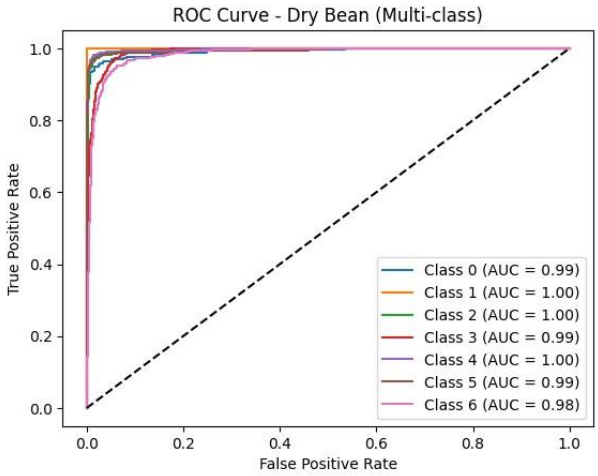
OUTPUT

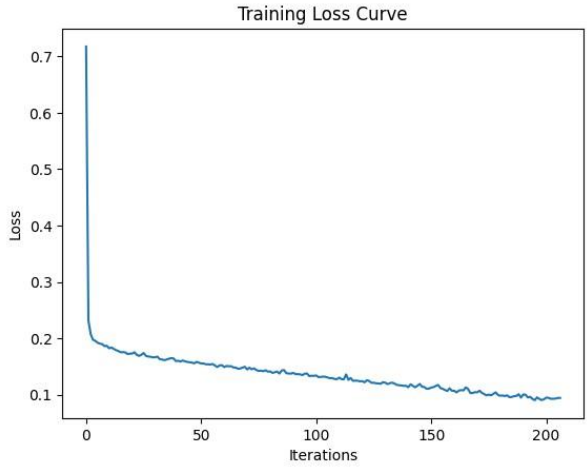
```
Dataset 1 - Dry Bean:
  Accuracy: 0.9243481454278369

  Classification Report:
              precision  recall  f1-score  support
Barbunya       0.94    0.90      0.92      261
    Bombay         1.00    1.00      1.00      117
    Cali           0.91    0.96      0.93      317
    Dermosan       0.90    0.92      0.91      671
    Horoz          0.97    0.95      0.96      408
    Seker          0.97    0.94      0.95      413
Sira           0.87    0.88      0.87      536
accuracy                         0.92     2723

Dataset 2 - Phishing Websites:
  Accuracy: 0.9674355495251018

  Classification Report:
         precision  recall  f1-score  support
-1      0.97    0.95      0.96      956
1      0.96    0.98      0.97     1255
accuracy                  0.97     2211

ROC AUC (Dry Bean) per class: 0.98-1.00
ROC AUC (Phishing): 0.995
```

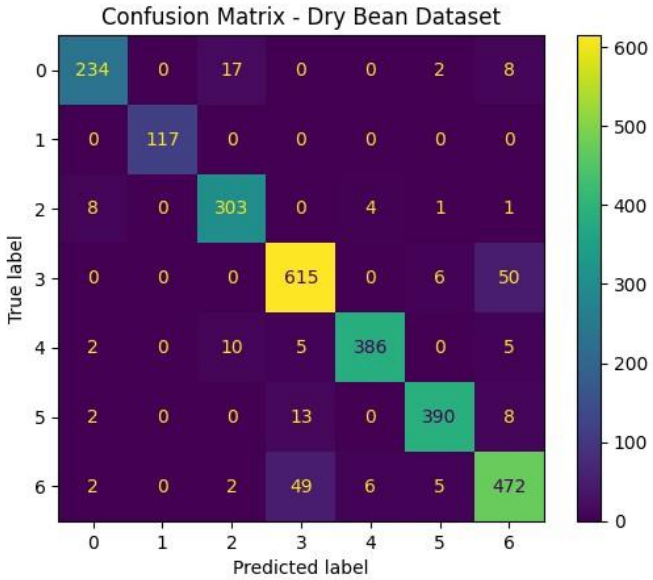

Training Loss Curve
Loss
Iterations


ROC Curve - Phishing Dataset
True Positive Rate
False Positive Rate
AUC = 0.995


Confusion Matrix - Phishing Dataset
True label
Predicted label
909
47
25
1230


ROC Curve - Dry Bean (Multi-class)
True Positive Rate
False Positive Rate
Class 0 (AUC = 0.99)
Class 1 (AUC = 1.00)
Class 2 (AUC = 1.00)
Class 3 (AUC = 0.99)
Class 4 (AUC = 1.00)
Class 5 (AUC = 0.99)
Class 6 (AUC = 0.98)


Training Loss Curve
Loss
Iterations


Confusion Matrix - Dry Bean Dataset
True label
Predicted label
234 0 17 0 0 2 8
0 117 0 0 0 0 0
8 0 303 0 4 1 1
0 0 0 615 0 6 50
2 0 10 5 386 0 5
2 0 0 13 0 390 8
2 0 2 49 6 5 472

RESULT

The MLP Neural Network achieved 92.43% accuracy on the Dry Bean dataset and 96.74% on the Phishing Websites dataset. High AUC values (0.98–1.00) across all classes confirm excellent discriminatory power of the neural network model.